AMAZON CLONE

Documentación Oficial del Proyecto

Versión del documento	1.0.0
Fecha de emisión	2026-10-11
Estado	En desarrollo
Autor	Dylan Dominguez Perez
Contacto	github.com/DylanDominguez
Audiencia	Equipo de desarrollo
Commit documentado	`9bb4aa8`

Documento destinado al equipo de desarrollo. La fuente de verdad es **docs/DOCUMENTACION.md**; este PDF es un entregable generado automáticamente.

Índice

1. Información del documento
    1.1 Historial de versiones del documento
2. Resumen ejecutivo
    2.1 Propósito
    2.2 Objetivos
    2.3 Alcance
3. Stack tecnológico
    3.1 Herramientas de documentación
4. Arquitectura del proyecto
    4.1 Estructura actual del repositorio
    4.2 Estructura objetivo (provisional)
    4.3 Modelo arquitectónico
5. Instalación y arranque
    5.1 Generar la documentación (disponible hoy)
    5.2 Ejecutar la aplicación (pendiente)
6. Flujo de trabajo del equipo
    6.1 Ramas
    6.2 Ciclo de una funcionalidad
    6.3 Convenciones de commit
7. Guía de desarrollo y convenciones
    7.1 Convenciones de código
    7.2 Estructura de un componente
    7.3 Reglas del equipo
8. Inventario de componentes
9. Rutas y navegación
10. Estado global y datos
11. Sistema de diseño
12. Roadmap / Plan de trabajo
13. Historial de cambios (Changelog)

v1.0.0 — 2026-10-11 — 9bb4aa8

14. Proceso de actualización de esta documentación

14.1 Cuándo actualizarlo

14.2 Procedimiento

14.3 Fuente de verdad

> **Cómo usar este documento.** Esta es la fuente oficial de información del proyecto. Todo lo que el equipo necesita saber (arquitectura, convenciones, estado del código y decisiones tomadas) se documenta aquí. El PDF se genera a partir de este archivo Markdown con `npm --prefix docs run generar`. Las secciones marcadas como **Pendiente** corresponden a funcionalidades aún no implementadas y se completarán en futuras versiones de este documento.

---

1. Información del documento

Campo	Valor
Proyecto	Amazon Clone with React
Versión del documento	1.0.0
Fecha de emisión	2026-10-11
Estado	En desarrollo (fase inicial)
Autor	Dylan Dominguez Perez
Audiencia	Equipo de desarrollo
Commit documentado	`9bb4aa8`
Repositorio	`Amazon-Clone-with-React` (rama `main`)

1.1 Historial de versiones del documento

Versión	Fecha	Commit	Responsable	Descripción
1.0.0	2026-10-11	`9bb4aa8`	Dylan Dominguez Perez	Creación de la documentación oficial: estructura base, stack declarado, guías iniciales y proceso de mantenimiento.

---

2. Resumen ejecutivo

Amazon Clone with React es una aplicación web frontend que replica la experiencia principal de compra de Amazon. El proyecto está concebido como una plataforma de comercio electrónico con catálogo de productos, búsqueda, carrito de compras y flujo de compra.

El objetivo técnico del proyecto es construir una interfaz basada en **componentes reutilizables**, con código mantenible y una arquitectura de frontend moderna.

2.1 Propósito

- Replicar la interfaz y los flujos principales de la tienda Amazon.
- Aplicar y consolidar prácticas modernas de desarrollo frontend con React.
- Mantener un código organizado, modular y fácil de mantener a lo largo del tiempo.

2.2 Objetivos

1. Interfaz responsiva y funcional en distintos tamaños de pantalla.
2. Arquitectura de componentes reutilizables y con responsabilidades claras.
3. Gestión predecible del estado de la aplicación (carrito, sesión, filtros).
4. Código fuente acompañado de esta documentación oficial, actualizada en cada avance.

2.3 Alcance

Pendiente de definición. El detalle de funcionalidades (features) del clon se definirá con el equipo y se documentará en la sección 12 (Roadmap). Mientras tanto, el alcance se limita a lo declarado en el README del proyecto.

---

3. Stack tecnológico

Tecnología	Rol	Justificación	Estado
⚛ React	Librería de UI	Arquitectura por componentes, ecosistema maduro	Declarada
🟨 JavaScript (ES6+)	Lógica de la aplicación	Lenguaje estándar del ecosistema React	Declarada
🎨 CSS3	Estilos y layout	Estilos responsivos sin dependencias adicionales	Declarada
⚡ Vite	Build tool y dev server	Arranque instantáneo y build optimizado	Declarada
📦 npm	Gestión de dependencias	Empaquetado oficial de Node.js	En uso
🟩 Node.js	Runtime de herramientas	Ejecución de scripts de build y documentación	≥ 24 (entorno actual)

> **Nota:** Las versiones exactas de React y Vite se confirmarán y documentarán en la versión 1.1.0 de este documento, cuando se inicialice el proyecto con Vite. Las dependencias reales se mantendrán en `package.json` y serán la fuente de verdad; esta tabla refleja el stack declarado.

3.1 Herramientas de documentación

Herramienta	Rol
Markdown (`DOCUMENTACION.md`)	Fuente de verdad del documento
`marked`	Conversión de Markdown a HTML
Puppeteer	Renderizado e impresión a PDF (A4)
`docs/generar-pdf.mjs`	Script de generación del PDF

---

4. Arquitectura del proyecto

4.1 Estructura actual del repositorio

```
Amazon-Clone-with-React/
├── .git/                    # Control de versiones
├── README.md                # Presentación del proyecto
└── docs/                    # Sistema de documentación oficial
    ├── .gitignore           # Excluye node_modules
    ├── package.json         # Dependencias de la documentación
    ├── DOCUMENTACION.md      # Fuente de la documentación (este archivo)
    ├── plantilla.css         # Estilos de impresión del PDF
    ├── generar-pdf.mjs       # Script generador del PDF
    └── pdf/                  # PDFs generados (entregables)
```

Estado: el código fuente de la aplicación aún no ha sido creado.

4.2 Estructura objetivo (provisional)

Cuando se inicialice la aplicación con Vite, se prevé la siguiente organización. Esta tabla se confirmará y ajustará en la versión 1.1.0 del documento.

```
Amazon-Clone-with-React/
├── index.html
├── package.json
├── vite.config.js
├── public/
└── src/
    ├── main.jsx             # Punto de entrada
    ├── App.jsx              # Raíz de la aplicación
    ├── components/          # Componentes reutilizables
    ├── pages/               # Vistas / rutas
    ├── hooks/               # Hooks personalizados
    ├── context/             # Estado global (React Context)
    ├── services/            # Acceso a datos / API
    ├── assets/              # Imágenes y recursos estáticos
    └── styles/              # Hojas de estilo
```

4.3 Modelo arquitectónico

- **Component-based:** la UI se compone de componentes pequeños, reutilizables y con una única responsabilidad.
- **Flujo de datos unidireccional:** el estado desciende mediante props; los eventos ascienden mediante callbacks.
- **Separación de capas:** presentación (`components/`), vistas (`pages/`), lógica (`hooks/` , `services/`) y estado (`context/`).

---

5. Instalación y arranque

5.1 Generar la documentación (disponible hoy)

```
# 1. Instalar dependencias (solo la primera vez)
npm --prefix docs install

# 2. Generar el PDF
npm --prefix docs run generar
```

El PDF se escribe en `docs/pdf/Documentacion-Oficial-v<versión>.pdf`.

5.2 Ejecutar la aplicación (pendiente)

Pendiente: los comandos definitivos se documentarán al inicializar el proyecto. Previsiblemente:

```
npm install      # Instalar dependencias
npm run dev      # Servidor de desarrollo (Vite)
npm run build    # Build de producción
npm run preview  # Vista previa del build
```

---

6. Flujo de trabajo del equipo

6.1 Ramas

Rama	Propósito
`main`	Estado estable y aprobado del proyecto
`feature/<nombre>`	Desarrollo de una funcionalidad nueva
`fix/<nombre>`	Corrección de un bug

6.2 Ciclo de una funcionalidad

1. Crear la rama desde `main`.
2. Desarrollar y probar localmente.
3. Actualizar la documentación si corresponde (ver sección 14).
4. Merge a `main` con mensaje de commit convencional.
5. Regenerar el PDF oficial si hubo cambios relevantes.

6.3 Convenciones de commit

Formato: `tipo: descripción corta` — tipos: `feat`, `fix`, `docs`, `style`, `refactor`, `chore`.

Ejemplo: `feat: agregar tarjeta de producto en el home`.

---

7. Guía de desarrollo y convenciones

7.1 Convenciones de código

Área	Convención
Archivos de componentes	`PascalCase.jsx` (ej. `ProductCard.jsx`)
Componentes	Funcionales + hooks
Variables y funciones	`camelCase`
Componentes y tipos	`PascalCase`
Constantes globales	`SCREAMING_SNAKE_CASE`
Estilos	Archivo CSS por componente o carpeta `styles/`
Props	Validar tipos cuando la complejidad lo justifique

7.2 Estructura de un componente

```
function Componente({ prop1, prop2 }) {
  return (
    <div className="componente">
      {/* markup */}
    </div>
  );
}
export default Componente;
```

7.3 Reglas del equipo

- Un componente = un archivo = una responsabilidad.
- No duplicar lógica: extraer a hook o utilidad compartida.
- Toda funcionalidad nueva debe quedar reflejada en este documento.
- Los comentarios explican el *por qué*, no el *qué*.

---

8. Inventario de componentes

Estado: pendiente — se poblará con el primer código fuente.

Componente	Ubicación	Props	Responsabilidad
—	—	—	—

9. Rutas y navegación

Estado: pendiente.

Ruta	Página	Descripción
`/`	—	—

10. Estado global y datos

Estado: pendiente. La estrategia de estado global (React Context, store, etc.) y las fuentes de datos se documentarán al implementarse.

11. Sistema de diseño

Estado: pendiente. Paleta de colores, tipografía, espaciados y patrones visuales replicados de Amazon se documentarán en esta sección.

12. Roadmap / Plan de trabajo

Pendiente de definición con el equipo. Esta sección contendrá las fases del proyecto y el estado de cada una.

Fase	Entregable	Estado
0	Documentación oficial del proyecto	✅ Completada
1	Inicialización del proyecto (Vite + React)	⬜ Pendiente
2	Por definir	⬜ Pendiente

13. Historial de cambios (Changelog)

v1.0.0 — 2026-10-11 — `9bb4aa8`

- Creación del sistema de documentación oficial (`docs/`).
- Estructura completa del documento: resumen ejecutivo, stack, arquitectura, guías de instalación y desarrollo, convenciones, flujos de trabajo Git.
- Secciones de inventario (componentes, rutas, estado, diseño) preparadas para poblar según avance el código.
- Script generador de PDF (`generar-pdf.mjs`) con portada, índice y paginación.
- Definición del proceso oficial de actualización de la documentación (sección 14).

14. Proceso de actualización de esta documentación

Este documento es **oficial** y se mantiene actualizado en cada avance relevante del proyecto.

14.1 Cuándo actualizarlo

- Al agregar una funcionalidad nueva.
- Al modificar la arquitectura, estructura de carpetas o stack.
- Al cambiar convenciones del equipo.
- Al definir o modificar el roadmap.

14.2 Procedimiento

1. **Analizar el cambio:** revisar el código fuente y ejecutar `git log` desde el `commit_base` indicado en la portada.
2. **Actualizar las secciones afectadas** de `docs/DOCUMENTACION.md`.
3. **Incrementar la versión** según semver del documento:
   - *mayor:* reestructuración general del documento.
   - *menor:* nuevas secciones o contenido sustancial.
   - *parche:* correcciones o actualizaciones menores.
4. **Registrar la entrada en el changelog** (sección 13) y actualizar `fecha` y `commit_base` en el front matter.
5. **Regenerar el PDF:** `npm --prefix docs run generar`.
6. **Verificar** el PDF generado en `docs/pdf/`.

14.3 Fuente de verdad

Información	Fuente de verdad
Contenido del documento	`docs/DOCUMENTACION.md`
Versión del documento	Front matter de `DOCUMENTACION.md`
Dependencias de la app	`package.json` de la raíz
Estado del código	Repositorio y su historial de Git